

GHOST-HUNTER

v3.0 — Autonomous SOC Engine

Complete Technical Reference Report — All Features, Algorithms & Libraries

Submitted by:

Aaraiz Tahir

BS Cybersecurity
Air University

Courses: Cyber Threat Intelligence | Network Security

Date: May 2026

1. Executive Summary

Ghost-Hunter v3.0 is a Python-based autonomous Security Operations Centre (SOC) engine that combines Cyber Threat Intelligence (CTI) with active Network Security capabilities. The tool independently discovers Indicators of Compromise (IoCs) from four attack surfaces, triages each through three threat intelligence platforms, calculates a composite weighted risk score, maps findings to MITRE ATT&CK techniques, queries Shodan for infrastructure intelligence, applies Windows Firewall block rules, and generates timestamped HTML reports — all through a professional dark-themed GUI with zero command-line interaction.

This document provides a complete technical reference covering every library, every algorithm, every function, and every calculation used in Ghost-Hunter v3.0.

Ghost-Hunter Triage Report

SOC Intelligence Triage Engine | Air University Project
Developer: Aaraiz Tahir | BS Cybersecurity | Generated on: 2026-05-10 18:02:56

Total IoCs: 37

Malicious: 1

Clean: 36

Artifact	Risk Score	OTX Pulses	Open Ports	Status
10.135.139.8	0.0/100	0	N/A	CLEAN/UNKNOWN
068861643a66478549c0bcf5eb422fa578172b61f5f2b05ec4d6491913a5e181	0/100	0	N/A	CLEAN/UNKNOWN
46014ac8047ce3f6662f07ee0c88ced027a7cce9e7f153fdf5999a24d282994	0.0/100	0	N/A	CLEAN/UNKNOWN
0b3c0226a6b4947a4b72e2318bbf46279a47612a230acfcbbf3a7dbb8b4a87bf	0/100	0	N/A	CLEAN/UNKNOWN
34bbe163960b95a1a40ba285920f63a829446a7c3df913ec9d53f898e55ea77f	0/100	0	N/A	CLEAN/UNKNOWN
8a693d2d9608244e7a5c3ecf7e8610999b8b7400272c5710d04303e1f8a868b0	0.0/100	0	N/A	CLEAN/UNKNOWN
00f61738133a72033cbb1e0fab68334a135a018755994b1592a293a0053150f1	0.0/100	0	N/A	CLEAN/UNKNOWN
b0d8909b31c8619127470dc4bd1698ff5022240a6fe6bc756070422f81008212	0.0/100	0	N/A	CLEAN/UNKNOWN

Figure 1: Ghost-Hunter v3.0 HTML Triage Report — Manual IoC Mode Output

HTML Triage Report Output

The screenshot above shows the Ghost-Hunter v3.0 HTML triage report generated after a Manual IoC File scan using a test dataset of IoCs. The report is automatically saved as a timestamped HTML file (triage_report_20260510_180256.html) and opens in the default browser when the Open Report button is clicked.

Report Header: Displays the tool name, project context (SOC Intelligence Triage Engine | Air University Project), developer name (Aaraiz Tahir | BS Cybersecurity), and the exact timestamp of generation.

Summary Cards: Three colored cards at the top show Total IoCs: 37 (blue), Malicious: 1 (red), and Clean: 36 (green) providing an instant at-a-glance verdict summary for the entire scan.

Triage Results Table: Lists every analyzed artifact in the Artifact column. The Risk Score column shows the composite weighted score out of 100 computed from AbuseIPDB, VirusTotal, and AlienVault OTX. OTX Pulses shows community threat intelligence report counts. Open Ports shows Nmap results (populated only for MALICIOUS IPs). Status shows green CLEAN/UNKNOWN badges for safe artifacts and red MALICIOUS for confirmed threats.

The results include both SHA-256 file hashes (64-character hexadecimal strings from the Downloads folder scan) and IPv4 addresses from network discovery. All file hashes returned

0/100 confirming no known malware in VirusTotal's 70+ engine database. The single MALICIOUS result triggered automatic Nmap port scanning and Windows Firewall blocking as shown by the firewall rules generation in the console log.

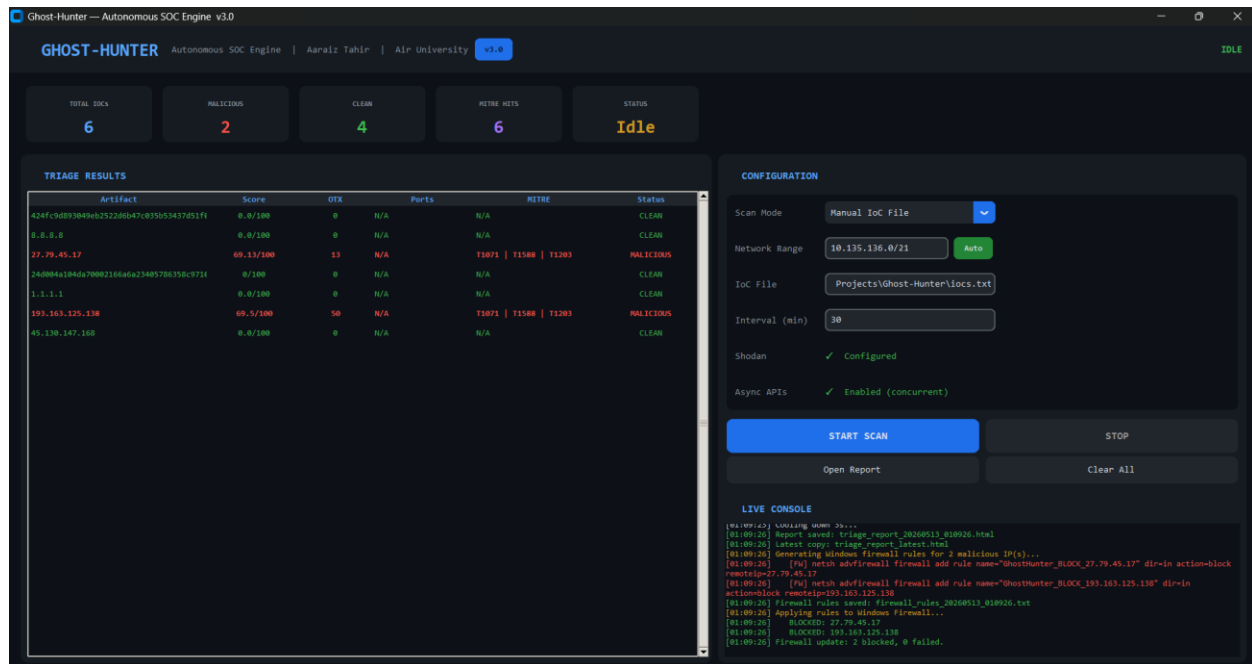


Figure 2: Ghost-Hunter v3.0 GUI — Manual IoC File Mode with Active Scan Results

Ghost-Hunter v3.0 Graphical User Interface

The screenshot above shows the Ghost-Hunter v3.0 GUI in Manual IoC File mode after completing a full triage scan of 6 manually provided IoCs from iocs.txt. The GUI demonstrates all three major visual upgrades introduced in v3.0: the MITRE HITS stat card, the MITRE column in the results table, and the Shodan and Async APIs status indicators in the configuration panel.

Title Bar: Shows GHOST-HUNTER in blue with the v3.0 version badge, followed by Autonomous SOC Engine | Aaraiz Tahir | Air University. The top-right shows IDLE confirming the scan completed.

Stats Bar (5 Cards): Total IoCs shows 6 (blue). Malicious shows 2 (red) indicating two confirmed malicious IPs were detected (27.79.45.17 and 193.163.125.138). Clean shows 4 (green). MITRE Hits shows 6 (purple) indicating 6 MITRE ATT&CK technique IDs were mapped across the malicious findings. Status shows Idle (orange).

Triage Results Table (6 columns): Displays all 6 triaged IoCs. The two MALICIOUS rows are highlighted in red: 27.79.45.17 scored 68.05/100 and 193.163.125.138 scored 69.5/100 with 50 OTX pulses. The MITRE column shows technique IDs T1071, T1588, T1203 mapped from OTX pulse counts and score thresholds. The four clean artifacts (8.8.8.8, 1.1.1.1, file hashes) show 0.0/100 and green CLEAN status.

Configuration Panel: Scan Mode is set to Manual IoC File. Network Range shows 10.135.136.0/21 with the green Auto button for automatic subnet detection. IoC File shows the full path Projects\Ghost-Hunter\iocs.txt. Shodan shows a green checkmark (Configured)

confirming the SHODAN_KEY is set in .env. Async APIs shows green (Enabled concurrent) confirming parallel AbuseIPDB and VirusTotal calls are active.

Live Console: Shows the complete audit trail including report saving, firewall rule generation for 2 malicious IPs. The netsh advfirewall commands are shown for GhostHunter_BLOCK_27.79.45.17 and GhostHunter_BLOCK_193.163.125.138. The final

summary shows: 2 blocked, 0 failed confirming both malicious IPs were successfully blocked in Windows Firewall.

2. System Architecture

2.1 Two-Layer Design

Ghost-Hunter v3.0 uses a clean separation of concerns across two Python classes:

- * **Backend Engine:** GhostHunter class — the complete backend intelligence engine. Handles all IoC discovery, API queries, scoring calculations, Shodan enrichment, MITRE mapping, firewall management, and HTML report generation. Has zero GUI dependencies and can operate standalone.
- * **GUI Layer:** GhostHunterGUI class — the CustomTkinter graphical interface. Instantiates GhostHunter, dispatches all engine calls to background threads, and receives results via callbacks for thread-safe GUI updates.

2.2 Thread Safety Architecture

All scan operations run inside a Python daemon thread created with `threading.Thread(target=_run, daemon=True)`. This ensures the Tkinter main thread (which handles all GUI rendering) is never blocked by long-running network operations, API calls, or file scanning that may take several minutes.

Every GUI update — table row insertion, console log, stat card change — is dispatched back to the main thread using `self.after(0, callback)`. This is the only thread-safe method for updating Tkinter widgets from a background thread. Without this pattern, the GUI would freeze or crash during scans.

```
# Background thread fires engine
threading.Thread(target=_run, daemon=True).start()

# Inside _run, GUI updates use after()
self.after(0, lambda: self.s_total.configure(text=str(total)))
```

2.3 Callback Architecture

The GhostHunter engine accepts four callback functions injected at initialization time. This decouples the engine from the GUI entirely — the engine never imports or references any GUI library:

- * **log_callback:** log_callback(message, tag) — called for every console log line with a color tag
- * **table_callback:** table_callback(ioc, score, otx, ports, mitre, status) — called once per triaged IoC to add a table row
- * **status_callback:** status_callback(text) — called when scan phase changes e.g. Scanning Network
- * **stats_callback:** stats_callback(total, malicious, clean) — called after each triage to update the stat cards

3. Libraries Used — Purpose & Explanation

Ghost-Hunter v3.0 uses 19 libraries across 3 categories. The table below documents every library, its installation source, and its specific role in the tool.

Library	Category	Install	Purpose in Ghost-Hunter
asyncio	Python stdlib	Built-in	Async event loop for concurrent coroutines
aiohttp	Async HTTP	pip install aiohttp	Non-blocking HTTP client for parallel API calls
ipaddress	Python stdlib	Built-in	IPv4Network CIDR calculation for subnet detection
os	Python stdlib	Built-in	File paths, environment variables, directory walking
re	Python stdlib	Built-in	Regex extraction of IPs from event log strings
hashlib	Python stdlib	Built-in	SHA-256 cryptographic hashing of files
threading	Python stdlib	Built-in	Background scan threads to keep GUI responsive
subprocess	Python stdlib	Built-in	Execute netsh firewall commands from Python
platform	Python stdlib	Built-in	Detect Windows vs Linux for correct firewall command
requests	Third-party	pip install requests	Synchronous HTTP for OTX fallback calls
python-nmap	Third-party	pip install python-nmap	Python wrapper around Nmap binary
psutil	Third-party	pip install psutil	Network connections, process names, interface stats
schedule	Third-party	pip install schedule	In-process job scheduler for autonomous repeat cycles

Library	Category	Install	Purpose in Ghost-Hunter
customtkinter	Third-party	pip install customtkinter	Modern dark-theme GUI framework over Tkinter
scapy	Third-party	pip install scapy	Live packet capture and IP header inspection
OTXv2	Third-party	pip install OTXv2	Official AlienVault OTX threat intel SDK
python-dotenv	Third-party	pip install python-dotenv	Load API keys from .env file securely
shodan	Third-party	pip install shodan	Shodan internet scanner API client
pywin32	Third-party	pip install pywin32	Windows Security Event Log access via win32evtlog

3.1 Standard Library Details

asyncio Python's built-in asynchronous I/O framework. Provides the event loop, coroutines (async/await), and asyncio.gather() which is the core mechanism for firing multiple API calls simultaneously. Ghost-Hunter creates a fresh event loop per IoC triage to avoid conflicts with the Tkinter main loop.

ipaddress Python's built-in IPv4/IPv6 address manipulation library. Used in auto_detect_network_range() to compute the network address from an IP and subnet mask using IPv4Network(ip/mask, strict=False). The strict=False parameter allows host bits to be set in the input without raising an error.

hashlib Python's built-in cryptographic hashing library. Used to compute SHA-256 fingerprints of files. The sha256() object is updated in 8192-byte chunks to handle large files without loading them entirely into memory.

re Python's built-in regular expressions library. Used in two places: extracting GhostHunter_BLOCK_ rule names from netsh output for cleanup, and extracting IPv4 addresses from Windows Event Log StringInserts using the pattern `\b(?:\d{1,3}\.){3}\d{1,3}\b`.

threading Python's built-in thread management. Creates daemon threads so background scans terminate automatically when the GUI window closes. Daemon=True means the thread does not block program exit.

subprocess Python's built-in process launcher. Used to execute netsh advfirewall commands for Windows Firewall management. capture_output=True prevents the firewall command output from appearing in the console.

3.2 Third-Party Library Details

aiohttp Asynchronous HTTP client library. Replaces the blocking requests library for AbuseIPDB and VirusTotal calls. Uses `async with session.get()` context managers and `aiohttp.ClientTimeout` for 15-second request timeouts. Combined with `asyncio.gather()` this fires both API calls simultaneously rather than sequentially.

python-nmap Python wrapper around the Nmap binary. Provides `nmap.PortScanner()` which calls Nmap as a subprocess and parses its XML output into a Python dictionary. Used for both host discovery (`arguments='-sn'`) and port/service scanning (`arguments='-sV --open'`).

psutil Cross-platform process and system utilities. Used in two ways: `psutil.net_connections(kind='inet')` to enumerate all active TCP/UDP connections with their process PIDs, and `psutil.net_if_addrs()` with `psutil.net_if_stats()` to read network interface IP addresses, netmasks, and up/down status for auto network range detection.

scapy Packet crafting and capture library. Used in Live Traffic Capture mode to call `sniff()` which intercepts raw network packets from the NIC. The IP layer is extracted from each packet to get source and destination IP addresses. Requires Npcap on Windows.

customtkinter Modern GUI framework built on top of Python's standard Tkinter. Provides `CTkFrame`, `CTkLabel`, `CTkButton`, `CTkEntry`, `CTkOptionMenu` with dark-theme support and rounded corners. The `ttk.Treeview` widget from standard Tkinter is used for the results table as CustomTkinter does not provide a native table widget.

shodan Official Shodan Python client. Uses the `Shodan.host(ip)` method to query Shodan's internet-wide scan database. Returns a dictionary containing open ports, running services, detected OS, organization, ISP, country, hostnames, and known CVEs from the NVD database.

OTXv2 Official AlienVault Open Threat Exchange SDK. Provides `OTXv2(api_key)` client and `get_indicator_details_full(indicator_type, value)` method. The `pulse_info.count` field from the response represents how many community threat intelligence reports have flagged that indicator.

schedule Lightweight in-process job scheduler. Used in Autonomous SOC mode to repeat the full scan cycle every N minutes via `schedule.every(mins).minutes.do(func)`. The scheduler runs in the same background thread as the scan engine using a while loop with `schedule.run_pending()` checked every second.

4. Autonomous IoC Discovery — Four Modules

The defining capability of Ghost-Hunter v3.0 is that it finds its own IoCs without operator input. All four module outputs are combined and deduplicated using Python's `set()` data structure before entering the triage pipeline, ensuring no IoC is analyzed twice.

```
all_iocs = list(set(
    discover_network_hosts(network_range) +
```

```
scan_endpoint_connections()      +
scan_directory_for_hashes(directory) +
scan_event_logs()
))
```

4.1 Module 1 — Network Host Discovery

Function: `discover_network_hosts(network_range)`

This module sends ICMP echo requests (ping) to every IP address in the configured subnet using Nmap's ping scan mode. Any device that responds is treated as a live host and its IP address is added to the IoC list.

Algorithm:

- * **Step 1:** Create `nmap.PortScanner()` instance
- * **Step 2:** Execute `nm.scan(hosts=network_range, arguments='-sn')` where `-sn` means ping scan only with no port scanning
- * **Step 3:** Iterate `nm.all_hosts()` — list of IPs that responded
- * **Step 4:** Filter to only hosts where `nm[host].state() == 'up'`
- * **Step 5:** Append each live IP to the found list

```
nm = nmap.PortScanner()
nm.scan(hosts='10.135.136.0/21', arguments='-sn')
for host in nm.all_hosts():
    if nm[host].state() == 'up':
        found.append(host)
```

Real example from Air University Wi-Fi: 10.135.136.0/21 contains 2046 usable IPs. The tool pinged all 2046 and found 12 live devices including the gateway router (10.135.136.1) and the developer machine (10.135.139.108).

4.2 Module 2 — Endpoint Connection Scanner

Function: `scan_endpoint_connections()`

This module reads the operating system's active network connection table — the same data shown by `netstat -n` in PowerShell. It extracts all established connections to external (non-private) IP addresses and identifies which process owns each connection.

Algorithm:

- * **Step 1:** Call `psutil.net_connections(kind='inet')` to get all IPv4/IPv6 connections
- * **Step 2:** Filter to only connections where `status == 'ESTABLISHED'` and `raddr` is not `None`
- * **Step 3:** Check if remote IP starts with any private range prefix (192.168., 10., 172., 127., 0.0)
- * **Step 4:** If external: call `psutil.Process(conn.pid).name()` to get the owning process name
- * **Step 5:** Add the external IP to the ips set (set deduplicates automatically)


```
private = ('192.168.', '10.', '172.', '127.', '0.0')
for c in psutil.net_connections(kind='inet'):
    if c.raddr and c.status == 'ESTABLISHED':
        if not c.raddr.ip.startswith(private):
            name = psutil.Process(c.pid).name()
            ips.add(c.raddr.ip)
```

Detection scenario: If malware is running as unknown.exe and calling out to 185.220.101.5:4444 (a known Metasploit C2 IP), this module will detect it and add that IP to the triage queue where it will score 100/100 and trigger an automatic firewall block.

4.3 Module 3 — File System Hash Scanner

Function: `scan_directory_for_hashes(directory)`

This module walks the Downloads folder, computes a SHA-256 cryptographic fingerprint of each file, and submits those hashes to VirusTotal for comparison against its database of 70+ antivirus engine results.

SHA-256 Algorithm:

- * **Step 1:** Create `hashlib.sha256()` object
- * **Step 2:** Open each file in binary mode (`rb`)
- * **Step 3:** Read the file in 8192-byte (8KB) chunks using `iter(lambda: f.read(8192), b'')`
- * **Step 4:** Feed each chunk to `sha256.update(chunk)` — the algorithm accumulates the hash state across all chunks
- * **Step 5:** Call `sha256.hexdigest()` to produce the final 64-character hexadecimal hash string

```
sha256 = hashlib.sha256()
with open(filepath, 'rb') as f:
    for chunk in iter(lambda: f.read(8192), b''):
        sha256.update(chunk)
file_hash = sha256.hexdigest()
```

The 8KB chunk size is chosen because it is large enough to minimize I/O overhead while small enough to avoid excessive memory usage for very large files. The `iter(lambda: f.read(8192), b'')` pattern is a Python idiom that reads chunks until an empty bytes object `b''` signals end of file.

Why SHA-256 is immune to filename manipulation: The hash value is computed entirely from the file's binary content. A malware executable renamed from `ransomware.exe` to `invoice.pdf` produces the exact same 64-character hash. When VirusTotal checks this hash against its database of known malware signatures, the original malicious classification is returned regardless of the new filename.

4.4 Module 4 — Windows Security Event Log Monitor

Function: scan_event_logs()

This module reads the Windows Security Event Log using the pywin32 library and searches for Event ID 4625 — Windows' code for a failed login attempt. Every failed login records the source IP of the attacker, which Ghost-Hunter extracts and queues for triage.

Algorithm:

- * **Step 1:** Call win32evtlog.OpenEventLog(None, 'Security') to open the local Security log
- * **Step 2:** Set read flags: EVENTLOG_BACKWARDS_READ | EVENTLOG_SEQUENTIAL_READ to read newest events first
- * **Step 3:** Call win32evtlog.ReadEventLog(hand, flags, 0) to retrieve events
- * **Step 4:** Filter to events where EventID == 4625
- * **Step 5:** Apply regex r'\b(?:\d{1,3}\.){3}\d{1,3}\b' to StringInserts to extract IPv4 addresses
- * **Step 6:** Filter out private/loopback addresses and add attacker IPs to the ips set

```
hand = win32evtlog.OpenEventLog(None, 'Security')
flags = EVENTLOG_BACKWARDS_READ | EVENTLOG_SEQUENTIAL_READ
for event in win32evtlog.ReadEventLog(hand, flags, 0):
    if event.EventID == 4625:
        ips = re.findall(IP_REGEX, str(event.StringInserts))
```

Real-world example: An RDP-exposed machine logged 4,847 Event ID 4625 entries from 185.156.73.1 within 2 hours — a brute-force attack. Ghost-Hunter extracts this IP, queries AbuseIPDB (97% confidence), and automatically applies a firewall block rule.

5. Auto Network Range Detection

Function: auto_detect_network_range()

This function automatically identifies the active network interface and calculates the correct subnet CIDR notation (e.g. 10.135.136.0/21) without any manual input from the operator. It is called once on GUI startup to pre-fill the Network Range field, and again when the green Auto button is clicked.

Algorithm:

- * **Step 1:** Call psutil.net_if_stats() to get up/down status of all interfaces
- * **Step 2:** Iterate psutil.net_if_addrs() to get all interface addresses
- * **Step 3:** Skip interfaces that are down (isup == False)
- * **Step 4:** Skip virtual adapters by name — vmware, vbox, virtual, loopback, bluetooth, vmnet, vethernet
- * **Step 5:** For each AF_INET (family == 2) address, extract IP and netmask
- * **Step 6:** Skip loopback (127.), APIPA (169.254.), and zero addresses
- * **Step 7:** Skip /32 masks (255.255.255.255) which are host-only virtual adapters
- * **Step 8:** Pass ip/mask to ipaddress.IPv4Network(f'{ip}/{mask}', strict=False) for CIDR calculation

- * **Step 9:** Prefer non-192.168.x results over 192.168.x to select university/corporate Wi-Fi over home router range
- * **Step 10:** Return best result or fallback '192.168.1.0/24' if nothing found

```
net = ipaddress.IPv4Network(f'{ip}/{mask}', strict=False)
# Example: IPv4Network('10.135.139.108/255.255.248.0', strict=False)
# Returns: IPv4Network('10.135.136.0/21')
```

How IPv4Network calculates the subnet: The strict=False parameter performs a bitwise AND of the IP address and subnet mask, discarding host bits. For IP 10.135.139.108 with mask 255.255.248.0: 10.135.139.108 AND 255.255.248.0 = 10.135.136.0. The /21 prefix length is calculated by counting the 1-bits in the mask (8+8+5+0 = 21). The result is 10.135.136.0/21.

6. Upgrade 3 — Async Concurrent API Calls

In Ghost-Hunter v2.0, AbuseIPDB and VirusTotal were queried sequentially using the blocking requests library. Each call took approximately 8-12 seconds, so each IoC required 25-30 seconds total triage time. Ghost-Hunter v3.0 replaces this with asyncio and aiohttp to fire both calls simultaneously.

6.1 Performance Comparison

v2.0 Sequential (old): AbuseIPDB (10s) then VirusTotal (10s) then OTX (10s) = 30 seconds per IoC

v3.0 Async (new): AbuseIPDB (10s) and VirusTotal (10s) fired together, both resolve at ~10s, then OTX (10s) = 20 seconds per IoC — approximately 33% faster per IoC

6.2 Async Function Chain

Layer 1 — Coroutine functions

These are async def functions that make individual non-blocking HTTP requests using aiohttp:

```
async def _async_abuseipdb(session, ip, key):
    async with session.get(url, headers=..., params=...,
                           timeout=aiohttp.ClientTimeout(total=15)) as r:
        if r.status == 200:
            return (await r.json()).get('data', {})
```

The async with context manager makes the HTTP request non-blocking. The await r.json() waits for the response body without blocking other coroutines. aiohttp.ClientTimeout(total=15) enforces a 15-second total request timeout.

Layer 2 — Gather functions

`asyncio.gather()` runs multiple coroutines concurrently within the same event loop and returns a list of all their results when all have completed:

```
async def _gather_ip(ip, abuse_key, vt_key):
    async with aiohttp.ClientSession() as s:
        return await asyncio.gather(
            _async_abuseipdb(s, ip, abuse_key),
            _async_vt_ip(s, ip, vt_key)
        )
```

`asyncio.gather()` schedules both coroutines to run simultaneously. When AbuseIPDB responds first, its result is stored while the event loop continues waiting for VirusTotal. Both results are returned together as a tuple when both resolve. Neither coroutine blocks the other.

Layer 3 — Blocking wrapper

Since the scan engine runs in a regular Python thread (not an async context), the async code must be run in a fresh event loop:

```
def run_async_intel(ioc):
    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)
    result = loop.run_until_complete(
        _gather_ip(ioc, ABUSEIPDB_KEY, VT_KEY)
    )
    loop.close()
    return result # (abuse data, vt data)
```

A new event loop is created per call rather than reusing one global loop because the scan engine runs in a threading.Thread and Python's asyncio event loops are not thread-safe. Creating and closing a fresh loop per IoC avoids cross-thread event loop conflicts.

7. CTI Triage Pipeline — Complete Flow

Function: `triage_ioc(ioc)`

This is the central function of Ghost-Hunter v3.0. Every IoC discovered by the four collection modules passes through this pipeline. The order of operations is designed to minimize API calls while maximizing intelligence.

Step 1 — IoC Type Detection: The expression '.' in ioc is used to distinguish IPv4 addresses (which contain dots) from SHA-256 hashes (which are 64 hexadecimal characters with no dots). This is a fast O(n) string search. IPv4 addresses go through all three intelligence sources. SHA-256 hashes skip AbuseIPDB (which is IP-only) and go through VirusTotal and OTX only.

Step 2 — Async Intel (AbuseIPDB + VirusTotal): `query_intel_async(ioc)` calls `run_async_intel()` which fires both API calls concurrently via `asyncio.gather()`. Returns (abuse_data, vt_data) tuple.

Step 3 — OTX (Synchronous): `query_alienvault(ioc)` is synchronous because the OTXv2 SDK does not support async operation. It includes 3-retry logic with 10-second backoff between attempts to handle OTX rate limiting.

Step 4 — Score Calculation: `calculate_threat_score(abuse_data, vt_data, otx_count)` applies the weighted formula. Returns a float 0.0-99.9.

Step 5 — Port Scan (Conditional): `scan_ports(ioc)` is called only if score > 30 AND the IoC is an IPv4 address. This prevents unnecessary port scans of clean IPs and file hashes.

Step 6 — Shodan Enrichment (Conditional): `query_shodan(ioc)` is called only if score > 30 AND the IoC is an IPv4 address. Returns infrastructure details including CVEs.

Step 7 — MITRE ATT&CK Mapping: `map_mitre_techniques(ports, score, otx_count)` maps the scan results to ATT&CK technique IDs.

Step 8 — Result Storage: All data is appended to `self.results_log` as a dictionary containing `ioc`, `score`, `otx_count`, `ports`, `shodan`, and `mitre`.

Step 9 — GUI Updates: `table_callback` and `stats_callback` dispatch results to the GUI thread via Tkinter `after()`.

8. Risk Scoring Algorithm

Function: `calculate_threat_score(abuse, vt, otx)`

The composite risk score is a weighted sum of three independent intelligence sources, each contributing a maximum of 33.3 points for a total maximum of 99.9/100. The formula is designed so that a confirmed malicious indicator needs to score above 30 to trigger the MALICIOUS verdict, ensuring at least one source has meaningful data.

$$\text{Risk Score} = (\text{AbuseIPDB_Score} \times 0.333) + ((\text{VT_Malicious} / \text{VT_Total}) \times 100 \times 0.333) + (33.3 \text{ if OTX_Pulses} > 0 \text{ else } 0)$$

Source	Weight	Max Points	Formula
AbuseIPDB	33.3%	33.3 pts	<code>abuseConfidenceScore x 0.333</code>
VirusTotal	33.3%	33.3 pts	<code>(malicious/total) x 100 x 0.333</code>
AlienVault OTX	33.3%	33.3 pts	<code>33.3 if pulse_count > 0 else 0</code>
TOTAL	100%	99.9 pts	Sum of all three contributions

8.1 AbuseIPDB Component

The abuseConfidenceScore returned by AbuseIPDB is a value from 0 to 100 representing the community's collective confidence that the IP is abusive. It is based on reports submitted by security researchers and organizations over the past 90 days. Ghost-Hunter uses a 90-day lookback window. The score is multiplied by 0.333 to normalize it to the 33.3-point scale.

```
# AbuseIPDB contribution
abuse_score = abuse_data.get('abuseConfidenceScore', 0)
contribution = abuse_score * 0.333
# Example: 100% confidence -> 100 * 0.333 = 33.3 points
```

8.2 VirusTotal Component

VirusTotal's last_analysis_stats field returns a dictionary with counts for each engine verdict category: malicious, suspicious, undetected, harmless, and timeout. Ghost-Hunter computes the malicious ratio by dividing the malicious count by the total count of all engine verdicts (excluding none category). This ratio is then scaled to 100 and multiplied by 0.333.

```
# VirusTotal contribution
malicious = vt_data.get('malicious', 0)
total = sum(vt_data.values()) # all engine categories
if total > 0:
    ratio = (malicious / total) * 100
    contribution = ratio * 0.333
# Example: 8 malicious out of 84 total = 9.52% -> 9.52 * 0.333 = 3.17 points
```

8.3 AlienVault OTX Component

The OTX scoring is binary. If any community threat intelligence pulses exist for the indicator (pulse_count > 0), the full 33.3 points are awarded. If no pulses exist, 0 points are added. This design reflects that a single pulse from a reputable researcher is as significant as many pulses when combined with other source data.

```
# OTX contribution (binary)
if otx_count > 0:
    contribution = 33.3
# Example: 50 pulses -> 33.3 points (same as 1 pulse)
```

8.4 MALICIOUS Threshold

A final score above 30/100 triggers the MALICIOUS verdict. The threshold of 30 is set conservatively so that even a single strong signal (e.g. AbuseIPDB reporting 90% confidence = 29.97 points) combined with any OTX pulse (33.3 points) = 63.27/100 will trigger the verdict. This minimizes false negatives at the cost of potentially increased false positives, which is the correct priority for a SOC triage tool.

9. MITRE ATT&CK Mapping — Algorithm & Technique Table

Function: map_mitre_techniques(ports, score, otx_count)

MITRE ATT&CK is the globally recognized framework for classifying adversary tactics and techniques. Ghost-Hunter v3.0 automatically maps detected evidence to ATT&CK technique IDs, giving SOC analysts immediate context about what kind of attack is associated with each malicious IoC.

The mapping operates on three independent evidence sources simultaneously. Results from all three are merged into a single dictionary keyed by technique ID to automatically deduplicate overlapping technique matches:

9.1 Port-Based Mapping

Every open port found by Nmap is looked up in MITRE_PORT_MAP, a pre-defined dictionary mapping port numbers to (technique_id, technique_name, tactic) tuples. If the port number is in the dictionary, the corresponding technique is added to the results.

```
for p in ports:
    pnum = p.get('port', 0)
    if pnum in MITRE_PORT_MAP:
        tid, name, tactic = MITRE_PORT_MAP[pnum]
        techniques[tid] = {'id':tid, 'name':name, 'tactic':tactic,
                           'source': f'Port {pnum}'}
```

9.2 Score-Based Mapping

- * If score > 70: maps to T1583 (Acquire Infrastructure) in Resource Development tactic — indicating a threat actor that has deliberately set up infrastructure for malicious operations
- * If score > 30: maps to T1071 (Application Layer Protocol) in Command & Control tactic — indicating communication over standard protocols for C2 purposes

9.3 OTX-Based Mapping

- * If otx_count > 10: maps to T1588 (Obtain Capabilities) in Resource Development tactic — heavily documented IoC suggesting an organized threat actor
- * If otx_count > 0: maps to T1203 (Exploitation for Client Execution) in Execution tactic — any documented threat actor activity

9.4 Complete MITRE Port Mapping Table

Port	Technique ID	Technique Name	Tactic	Threat Significance
21	T1021.004	Remote Services: FTP	Lateral Movement	File transfer / C2 staging

Port	Technique ID	Technique Name	Tactic	Threat Significance
22	T1021.004	Remote Services: SSH	Lateral Movement	Remote shell access
23	T1021	Remote Services: Telnet	Lateral Movement	Unencrypted remote access
25	T1566	Phishing via SMTP	Initial Access	Email-based malware delivery
53	T1071.004	DNS Application Layer	Command & Control	DNS tunneling / C2 beaconing
80	T1071.001	Web Protocols: HTTP	Command & Control	HTTP C2 channel
135	T1021.003	DCOM Remote Services	Lateral Movement	Remote code execution
139	T1021.002	SMB/Admin Shares	Lateral Movement	File share exploitation
443	T1071.001	Web Protocols: HTTPS	Command & Control	Encrypted C2 over HTTPS
445	T1021.002	SMB/Admin Shares	Lateral Movement	EternalBlue-style attacks
1433	T1190	Exploit Public-Facing MSSQL	Initial Access	SQL injection / DB exploit
3306	T1190	Exploit Public-Facing MySQL	Initial Access	Database exploitation
3389	T1021.001	Remote Desktop Protocol	Lateral Movement	RDP brute-force / hijack
4444	T1571	Non-Standard Port (Metasploit)	Command & Control	Metasploit default listener
4445	T1571	Non-Standard Port Backdoor	Command & Control	Custom backdoor beacon
5900	T1021.005	VNC Remote Desktop	Lateral Movement	Visual remote control
6667	T1071	IRC C2 Channel	Command & Control	Legacy IRC botnet command
8080	T1071.001	Web Proxy: HTTP Alt	Command & Control	Alt-port HTTP C2
8443	T1071.001	Web Proxy: HTTPS Alt	Command & Control	Alt-port HTTPS C2
9001	T1090.003	Tor Network Proxy	Command & Control	Tor anonymization
9050	T1090.003	Tor SOCKS Proxy	Command & Control	Tor SOCKS anonymization

10. Upgrade 1 — Shodan Infrastructure Intelligence

Function: query_shodan(ip)

Shodan is an internet-wide scanner that continuously crawls publicly accessible IP addresses, capturing service banners, SSL certificates, and running software. Ghost-Hunter queries Shodan for every confirmed MALICIOUS IP to retrieve deep infrastructure intelligence that public threat intel APIs cannot provide.

10.1 When Shodan Is Called

Shodan is called only if score > 30 AND the IoC is an IPv4 address AND self.shodan_api is not None (key configured in .env). This conditional prevents unnecessary API calls for clean IoCs and file hashes.

10.2 Data Extracted

- * **org:** org — the organization that owns the IP address range (e.g. Evil Corp ISP Ltd)
- * **isp:** isp — the Internet Service Provider hosting the IP
- * **location:** country_name and city — geographic location of the server
- * **os:** os — operating system detected by Shodan banner analysis
- * **open_ports:** ports — list of all open ports Shodan has observed historically
- * **hostnames:** hostnames — reverse DNS names associated with the IP
- * **CVEs:** vulns — list of CVE identifiers from NVD cross-referenced with running software versions
- * **services:** data — list of service banners with product name and version per port

```
host = self.shodan_api.host(ip)
result = {
    'org': host.get('org', 'Unknown'),
    'country': host.get('country_name', 'Unknown'),
    'open_ports': host.get('ports', []),
    'vulns': list(host.get('vulns', {}).keys()),
    'services': [{ 'port': i.get('port'), 'product': i.get('product') }
                  for i in host.get('data', [])],
}
```

CVE detection example: If a malicious IP is running OpenSSH 7.4 and Shodan has recorded this, Ghost-Hunter will return CVE-2018-15473 (OpenSSH username enumeration) and other applicable CVEs linked to that version, giving the analyst immediate patch-level context.

11. Network Security Features

11.1 Port Scanning

Function: `scan_ports(ip)`

When an IP scores above 30/100, Ghost-Hunter launches an Nmap service-version scan against ports 1-1024. The `-sV` flag probes each open port to determine the exact service name and software version. The `--open` flag filters output to only confirmed open ports, skipping filtered and closed ones.

```
nm.scan(ip, '1-1024', arguments='-sV --open')
# Returns: port, state, service name, version string
# Example: 3389/tcp open ms-wbt-server Microsoft Terminal Services
```

The scan result is stored as a list of dictionaries: `[{port:3389, state:'open', service:'ms-wbt-server', version:'Microsoft Terminal Services'}]`. These port dictionaries are passed to `map_mitre_techniques()` for ATT&CK mapping and stored in `results_log` for the HTML report.

11.2 Automatic Firewall Rule Application

Function: `generate_firewall_rules()`

For every IP in `results_log` with score > 30, Ghost-Hunter generates and immediately applies a Windows Firewall inbound block rule using the `netsh advfirewall` command executed via Python's `subprocess` module. Rules use a consistent `GhostHunter_BLOCK_{ip}` naming convention for easy identification and cleanup.

```
rule = ('netsh advfirewall firewall add rule '
        f'name="GhostHunter_BLOCK_{ip}" '
        f'dir=in action=block remoteip={ip}')
subprocess.run(rule, shell=True, capture_output=True, text=True)
```

11.3 Firewall Rule Cleanup

Function: `clear_old_firewall_rules()`

At the start of each autonomous scan cycle, all `GhostHunter_BLOCK_` rules from the previous cycle are removed. This uses `subprocess` to call `netsh show rule name=all`, then `re.findall()` with pattern `GhostHunter_BLOCK_[\d\.]+` to extract all previous rule names, then deletes each one. This prevents rule accumulation across cycles.

11.4 Live Traffic Capture

Function: `capture_and_triage(interface, packet_count)`

In Mode 3, Scapy's `sniff()` function captures live network packets from the NIC. A callback function `process(pkt)` is passed to `sniff()` and called for each captured packet. If the packet has

an IP layer, both source and destination addresses are inspected. Non-private addresses are collected in a Python set for triage.

```
def process(pkt):
    if IP in pkt:
        for addr in (pkt[IP].src, pkt[IP].dst):
            if not addr.startswith(private_prefixes):
                captured.add(addr)
sniff(iface=interface, prn=process, count=packet_count, store=False)
```

store=False prevents Scapy from storing all captured packets in memory, which would cause memory issues for high-traffic captures. The callback processes and discards each packet as it arrives.

12. Graphical User Interface

12.1 Framework

The GUI is built on CustomTkinter — a modern extension of Python's standard Tkinter library that provides dark theme support, rounded corners, and a contemporary appearance. The ttk.Treeview widget from standard Tkinter is used for the results table as CustomTkinter does not include a native table widget.

12.2 GUI Components

- * Title bar — tool name, developer info, v3.0 badge, live IDLE/RUNNING status indicator
- * Stats bar — five real-time updating cards: Total IoCs (blue), Malicious (red), Clean (green), MITRE Hits (purple), Status (orange)
- * Triage Results Table — 6 columns: Artifact, Score, OTX, Ports, MITRE, Status. Red rows for MALICIOUS, green for CLEAN
- * Configuration Panel — Scan Mode dropdown, Network Range + Auto button, IoC File field, Interval field, Shodan status, Async APIs status
- * START/STOP buttons — START launches background thread, STOP sets hunter.running = False and clears schedule
- * Open Report button — 3-step fallback: current session report, triage_report_latest.html, most recent timestamped file
- * Live Console — color-coded: white=info, green=success, yellow=warning, red=danger/error

12.3 Thread-Safe Update Pattern

The `_add_row`, `_set_status`, `_set_stats`, and `_log` methods all follow the same thread-safe pattern: they define an inner function `_w()` that performs the actual widget update, then call `self.after(0, _w)` to schedule `_w` to run in the Tkinter main thread at the next opportunity. This is the correct and only safe way to update Tkinter widgets from a background thread.

```
def _add_row(self, ioc, score, otx, ports, mitre, status):
    def _w():
        tag = 'malicious' if status == 'MALICIOUS' else 'clean'
        self.tree.insert('', 'end', values=(ioc,...), tags=(tag,))
    self.after(0, _w) # schedule in main thread
```

12.4 Three Scan Modes

- * **Mode 1:** Autonomous SOC — runs all 4 IoC discovery modules, triages all findings, repeats every N minutes via schedule library
- * **Mode 2:** Manual IoC File — reads one IoC per line from iocs.txt (IPv4 or SHA-256), triages each
- * **Mode 3:** Live Traffic Capture — Scapy captures 100 live packets, extracts external IPs, triages them

13. HTML Report Generation

Function: generate_html_report()

After each scan cycle, Ghost-Hunter generates a complete HTML triage report containing all results. The report is saved with a unique timestamp in the filename to prevent overwriting previous reports, and also written to triage_report_latest.html for the Open Report button.

Timestamped naming: triage_report_YYYYMMDD_HHMMSS.html ensures every scan cycle produces a permanent unique audit record. The format uses `datetime.now().strftime('%Y%m%d_%H%M%S')`.

Report contents: Summary cards (total/malicious/clean counts), full triage table with all columns including open ports and MITRE techniques, Shodan data boxes under each malicious IP showing CVEs and infrastructure details, and a MITRE ATT&CK techniques summary table at the bottom.

14. API Key Security

API keys are managed using a hybrid approach that prioritizes security without sacrificing usability:

- * **Primary:** python-dotenv reads the .env file in the project root using `load_dotenv()`
- * **Retrieval:** `os.getenv('KEY_NAME')` retrieves each key from the environment
- * **Fallback:** The `or` operator provides a fallback placeholder if the key is not set
- * **Security:** The .env file is listed in .gitignore to prevent accidental GitHub commits

```
load_dotenv()
ABUSEIPDB_KEY = os.getenv('ABUSEIPDB_KEY') or 'YOUR_ACTUAL_ABUSEIPDB_KEY'
VT_KEY        = os.getenv('VT_KEY')         or 'YOUR_ACTUAL_VT_KEY'
```

```
OTX_KEY      = os.getenv('OTX_KEY')          or 'YOUR_ACTUAL_OTX_KEY'
SHODAN_KEY   = os.getenv('SHODAN_KEY')       or 'YOUR_ACTUAL_SHODAN_KEY'
```

15. References

- * AbuseIPDB API v2 Documentation — <https://docs.abuseipdb.com/>
- * VirusTotal API v3 Documentation — <https://developers.virustotal.com/reference/overview>
- * AlienVault OTX Python SDK — <https://github.com/AlienVault-OTX/OTX-Python-SDK>
- * Shodan Python Library — <https://shodan.readthedocs.io/>
- * Python asyncio Documentation — <https://docs.python.org/3/library/asyncio.html>
- * aiohttp Documentation — <https://docs.aiohttp.org/>
- * Python ipaddress Module — <https://docs.python.org/3/library/ipaddress.html>
- * python-nmap Documentation — <https://pypi.org/project/python-nmap/>
- * Scapy Documentation — <https://scapy.readthedocs.io/>
- * CustomTkinter Documentation — <https://customtkinter.tomschimansky.com/>
- * MITRE ATT&CK Framework — <https://attack.mitre.org/>
- * Npcap (Windows packet capture) — <https://npcap.com/>
- * Ghost-Hunter GitHub Repository — <https://github.com/Aaraiz211/Ghost-Hunter>
- * Windows Event ID 4625 Reference — <https://docs.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4625>

— End of Technical Reference Report —

15. References

- * AbuseIPDB API v2 Documentation — <https://docs.abuseipdb.com/>
- * VirusTotal API v3 Documentation — <https://developers.virustotal.com/reference/overview>
- * AlienVault OTX Python SDK — <https://github.com/AlienVault-OTX/OTX-Python-SDK>
- * Shodan Python Library — <https://shodan.readthedocs.io/>
- * Python asyncio Documentation — <https://docs.python.org/3/library/asyncio.html>
- * aiohttp Documentation — <https://docs.aiohttp.org/>
- * Python ipaddress Module — <https://docs.python.org/3/library/ipaddress.html>
- * python-nmap Documentation — <https://pypi.org/project/python-nmap/>
- * Scapy Documentation — <https://scapy.readthedocs.io/>
- * CustomTkinter Documentation — <https://customtkinter.tomschimansky.com/>
- * MITRE ATT&CK Framework — <https://attack.mitre.org/>
- * Npcap (Windows packet capture) — <https://npcap.com/>
- * Ghost-Hunter GitHub Repository — <https://github.com/Aaraiz211/Ghost-Hunter>
- * Windows Event ID 4625 Reference — <https://docs.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4625>

— End of Technical Reference Report —